CURRICULUM & STUDY BLUEPRINT

# 🗺️ Master Study Roadmap (All Topics Organized)

Complete Table of Contents, 12-Week Course Study Plan, and Full Module-by-Module Course Outlines (Modules 1–11)

## 1. Table of Contents


| Section | Module / Curriculum Title | Topic Range | Total Topics |
|---|---|---|---|
| Section 1 | **Table of Contents** | — | — |
| Section 2 | **Course Study Plan (12-Week Structured Schedule)** | Weeks 1–12 | 11 Modules |
| Module 1 | Generative AI Foundations & The 2026 Landscape | 1.1 – 1.4 | 4 |
| Module 2 | Transformer Architecture & Attention Mechanics | 2.1 – 2.7 | 7 |
| Module 3 | Tokenization & Context Windows | 3.1 – 3.4 | 4 |
| Module 4 | Model Scaling, Parameters & MoE | 4.1 – 4.4 | 4 |
| Module 5 | Decoding Strategies & Model Families | 5.1 – 5.6 | 6 |
| Module 6 | Production LLM APIs & State Management | 6.1 – 6.5 | 5 |
| Module 7 | Prompt Engineering & Security | 7.1 – 7.8 | 8 |
| Module 8 | Tool Calling & Agentic Workflows | 8.1 – 8.12 | 12 |
| Module 9 | Embeddings, Vector Databases & RAG Systems | 9.1 – 9.20 | 20 |
| Module 10 | Fine-Tuning, PEFT, Alignment & Distributed Training | 10.1 – 10.22 | 22 |
| Module 11 | Quantization & Inference Optimization | 11.1 – 11.16 | 16 |
| Total | **Complete Master Study Roadmap** | **1.1 – 11.16** | **108 Topics** |


## 2. Course Study Plan

| | Assigned Module(s) | Subtopics Covered & Study Milestones |
|---|---|---|
| **Week 1** | Module 1 & Module 3 | **1.1–1.4 & 3.1–3.4:** Generative vs. discriminative AI, autoregressive vs. diffusion, GPT-1 to GPT-5/Claude/Gemini timeline, 2026 app landscape, BPE/WordPiece/SentencePiece tokenization, subword dynamics, context budgets, and token cost calculation with tiktoken. |
| **Week 2** | Module 2 | **2.1–2.7:** Self-attention (Q/K/V, scaled dot-product), multi-head attention, positional encodings (sinusoidal, learned, RoPE, ALiBi), decoder-only vs. encoder-decoder vs. encoder-only, context extension (RoPE scaling, YaRN, NTK), KV-cache mechanics, and MQA/GQA memory savings. |

| | Assigned Module(s) | Subtopics Covered & Study Milestones |
|---|---|---|
| **Week 3** | **Module 4 & Module 5** | **4.1–4.4 & 5.1–5.6:** Parameter matrices (7B/70B/405B), Kaplan & Chinchilla scaling laws, Mixture-of-Experts (Mixtral, DeepSeek, GPT-4), decoding math (greedy, beam, top-k, top-p, temperature, min-p), frontier vs. open-weight families, reasoning models, and selection matrix. |
| **Week 4** | **Module 6** | **6.1–6.5:** Live LLM APIs (OpenAI chat completions, roles, SSE), streaming token generation over API/WebSocket, authentication, RPM/TPM rate limits, retry/backoff, token-based pricing and traffic budgeting, and stateless multi-turn conversation management. |
| **Week 5** | **Module 7** | **7.1–7.8:** Prompt anatomy & in-context learning, zero/few-shot selection, CoT, self-consistency, ToT, GoT, ReAct pattern, structured outputs (JSON mode, Pydantic constrained decoding), prompt chaining, DSPy automatic optimization, and prompt injection defenses. |
| **Week 6** | **Module 8** | **8.1–8.12:** Tool calling internals, schema design, OpenAI/Claude/Gemini function calling, parallel & multi-step tool chains, error handling & retries, sandboxing & human-approval gates, structured output integration, cost/latency, evaluation, and **Weekly Project (8.12): 5+ live tools assistant**. |
| **Week 7** | **Module 9 (Part I)** | **9.1–9.10:** Dense embeddings (OpenAI, Cohere, sentence-transformers), similarity metrics, chunking strategies, Matryoshka embeddings, Vector DB landscape (FAISS, Chroma, Pinecone, Weaviate, Qdrant, Milvus, pgvector), HNSW/IVF/PQ indexing, metadata filtering, DB selection, RAG lifecycle, and LlamaIndex core concepts. |
| **Week 8** | **Module 9 (Part II)** | **9.11–9.20:** LangChain & LCEL RAG pipelines, grounding & citation, hybrid search (BM25 + dense) & reranking, query transformation (HyDE, multi-query, decomposition), parent-document & sentence-window retrieval, Self-RAG & CRAG, GraphRAG, Multimodal RAG, failure modes, and **Weekly Project (9.20): Production RAG system**. |
| **Week 9** | **Module 10 (Part I)** | **10.1–10.12:** Fine-tuning vs. prompt engineering vs. RAG framework, full fine-tuning & catastrophic forgetting, instruction/chat datasets (ShareGPT/Alpaca), synthetic data generation, PEFT landscape, LoRA math & adapter merging, 4-bit quantization primer, QLoRA (NF4, double quantization, paged optimizers), VRAM math (7B/13B/70B), and single-GPU workflow. |
| **Week 10** | **Module 10 (Part II)** | **10.13–10.22:** RLHF pipeline (reward model + PPO), DPO, ORPO & KTO, building preference datasets, benchmarking against base models, overfitting diagnosis, cost-benefit review vs. RAG+prompt baseline, tooling (Axolotl, Unsloth, LLaMA-Factory), distributed training (DeepSpeed ZeRO, FSDP, gradient checkpointing), and model merging (SLERP, TIES, DARE, MergeKit). |

| | Assigned Module(s) | Subtopics Covered & Study Milestones |
|---|---|---|
| **Week 11** | **Module 11 (Part I)** | **11.1–11.8:** Quantization fundamentals (INT8/INT4, symmetric vs. asymmetric), PTQ vs. QAT, calibration datasets, quality/speed/memory trade-off curves, GPTQ Hessian-based weight quantization, group-size trade-offs, outlier weight impacts, and AutoGPTQ workflows. |
| **Week 12** | **Module 11 (Part II)** | **11.9–11.16:** AWQ theory & comparison against GPTQ, GGUF & llama.cpp levels (Q2_K through Q8_0, Q4_K_M, Q5_K_M), deployment target format selection, bitsandbytes 8-bit/NF4 inference, mixed-precision (fp16/bf16) vs. quantized inference, speculative decoding, and hardware considerations (24GB consumer GPU vs. A100/H100 vs. Apple Silicon/CPU). |

## 3. Complete Course Outlines

### Module 1: Generative AI Foundations & The 2026 Landscape

**1.1** Generative vs discriminative AI — precise distinction

**1.2** Autoregressive generation vs denoising/diffusion generation (text vs image/audio)

**1.3** Timeline: GPT-1→GPT-4/4o/5, BERT, T5, LLaMA, Claude, Gemini — what changed at each step

**1.4** GenAI application landscape 2026: copilots, agents, RAG systems, multimodal apps

### Module 2: Transformer Architecture & Attention Mechanics

**2.1** Self-attention mechanism: Query/Key/Value, scaled dot-product — formula walkthrough

**2.2** Multi-head attention — why multiple heads, what each head can learn

**2.3** Positional encoding: sinusoidal vs learned vs Rotary (RoPE) vs ALiBi — trade-offs

**2.4** Decoder-only (GPT-style) vs encoder-decoder (T5-style) vs encoder-only (BERT-style) — when each is used today

**2.5** Context window extension: RoPE scaling, YaRN, NTK-aware interpolation — how 8K becomes 128K/1M

**2.6** KV-cache mechanics and its role in inference cost/latency

**2.7** Multi-Query Attention (MQA) & Grouped-Query Attention (GQA) — why they exist, memory savings

### Module 3: Tokenization & Context Windows

**3.1** Tokenization deep dive: BPE, WordPiece, SentencePiece/Unigram — algorithmic differences

**3.2** What is a 'token' precisely — subword units, why 'strawberry' confuses LLMs

**3.3** Token limits & context window explained: how input + output share one budget

**3.4** Counting tokens in practice (tiktoken, provider tokenizers); cost implications of token count

## Module 4: Model Scaling, Parameters & MoE

**4.1** What '7B / 70B / 405B parameters' actually means — weight matrices, layers × heads × dim

**4.2** Parameter count vs capability — why more isn't always better (data quality, architecture)

**4.3** Scaling laws (Kaplan, Chinchilla) — compute-optimal training

**4.4** Mixture-of-Experts (MoE): sparse activation, active vs total parameters (Mixtral, DeepSeek, GPT-4 rumored architecture)

## Module 5: Decoding Strategies & Model Families

**5.1** Decoding strategies: greedy, beam search, top-k, top-p (nucleus), temperature — math and effect on output

**5.2** Repetition penalty, min-p, frequency/presence penalties

**5.3** Frontier model families compared: GPT (OpenAI), Claude (Anthropic), Gemini (Google) — architecture philosophy, context windows, strengths

**5.4** Open-weight families: LLaMA 3.x, Mistral/Mixtral, Qwen, DeepSeek, Gemma — licensing & deployment implications

**5.5** Reasoning models vs standard chat models (o-series/extended-thinking style models) — how they differ operationally

**5.6** Choosing a model: capability vs cost vs latency vs privacy decision matrix

## Module 6: Production LLM APIs & State Management

**6.1** Live LLM APIs — OpenAI: chat completions endpoint, message roles, streaming (SSE)

**6.2** Streaming token generation — how it works over an API/WebSocket

**6.3** Authentication, rate limits (RPM/TPM), retry/backoff strategies, error handling

**6.4** Token-based pricing — computing real request cost, budgeting for production traffic

**6.5** Multi-turn conversation state — how context is passed on every call (statelessness of the API)

## Module 7: Prompt Engineering & Security

**7.1** Prompt anatomy & in-context learning theory

**7.2** Zero-shot, few-shot, and example selection strategies

**7.3** Chain-of-Thought (CoT), self-consistency, Tree-of-Thought, Graph-of-Thought

**7.4** ReAct pattern (Reason + Act) — the conceptual bridge to tool calling

**7.5** Structured output generation: JSON mode, Pydantic/JSON-schema-constrained decoding

**7.6** Prompt chaining & task decomposition for multi-step pipelines

**7.7** Meta-prompting & automatic prompt optimization (DSPy overview)

**7.8** Prompt injection attacks, jailbreak patterns, and defensive prompting

## Module 8: Tool Calling & Agentic Workflows

**8.1** Tool/Function calling — how it works under the hood (schema injection, model-emitted call, execution, result round-trip)

**8.2** Designing tool schemas: naming, descriptions, parameter typing, required vs optional

**8.3** Single tool call end-to-end with OpenAI function calling

**8.4** Single tool call end-to-end with Anthropic Claude tool use and Gemini function calling — API differences

**8.5** Parallel tool calls & multi-tool selection (tool-choice strategies: auto/required/specific )

**8.6** Multi-step tool chains — sequential dependent calls, passing outputs between calls

**8.7** Error handling, retries, and validation failures in tool execution

**8.8** Security: sandboxing tool execution, permission scoping, human-approval gates for sensitive tools

**8.9** Combining structured output + tool calling in production pipelines

**8.10** Cost/latency implications of tool-heavy agent loops

**8.11** Testing & evaluating tool-calling reliability (schema adherence, correct tool selection)

**8.12** Weekly project: production-grade tool-calling assistant with 5+ live tools and full error handling

## Module 9: Embeddings, Vector Databases & RAG Systems

**9.1** Dense embedding models: OpenAI/Cohere/sentence-transformers embeddings

**9.2** Similarity metrics: cosine, dot product, Euclidean — when each matters

**9.3** Chunking strategies: fixed-size, recursive, semantic, structure-aware chunking

**9.4** Embedding dimensionality, storage cost, and Matryoshka/adaptive embeddings

**9.5** Vector DB landscape: FAISS, Chroma, Pinecone, Weaviate, Qdrant, Milvus, pgvector

**9.6** Indexing algorithms: HNSW, IVF, product quantization — accuracy/speed/memory trade-offs

**9.7** Metadata filtering, hybrid indexes, multi-tenant vector stores

**9.8** Choosing a vector DB for scale/cost/latency in production

**9.9** RAG architecture fundamentals: retriever + generator, the full request lifecycle

**9.10** LlamaIndex core concepts: documents, nodes, indices, query engines, retrievers

**9.11** LangChain retrieval chains, retrievers, and LCEL-based RAG pipelines

**9.12** Grounding & citation — attributing generated answers back to source chunks

**9.13** Advanced RAG: hybrid search (BM25 + dense), reranking (cross-encoders, Cohere Rerank)

**9.14** Query transformation: HyDE, multi-query retrieval, query decomposition

**9.15** Parent-document / small-to-big retrieval, sentence-window retrieval

**9.16** Self-RAG & Corrective RAG (CRAG) — retrieval-quality-aware generation

**9.17** GraphRAG & knowledge-graph-augmented retrieval

**9.18** Multimodal RAG (text + tables + images)

**9.19** RAG failure modes: retrieval miss, context dilution, lost-in-the-middle

**9.20** Weekly project: production RAG system over a real document set, with evaluation baked in

## Module 10: Fine-Tuning, PEFT, Alignment & Distributed Training

**10.1** Fine-tuning vs prompt engineering vs RAG — decision framework with cost/benefit analysis

**10.2** Full fine-tuning: mechanics, compute cost, catastrophic forgetting

**10.3** Dataset preparation: instruction format, chat templates, ShareGPT/Alpaca formats

**10.4** Data quality, deduplication, synthetic data generation with a teacher LLM

**10.5** Parameter-Efficient Fine-Tuning (PEFT) landscape: adapters, prefix-tuning, prompt-tuning, P-Tuning v2

**10.6** Low-Rank Adaptation (LoRA): mathematical formulation, rank (r) and alpha, target-module selection

**10.7** LoRA merging back into base weights vs keeping adapters separate (multi-adapter serving)

**10.8** Choosing rank/alpha/target modules for a given task and model size

**10.9** Quantization primer for fine-tuning: why 4-bit training is possible

**10.10** QLoRA: 4-bit NormalFloat (NF4), double quantization, paged optimizers — mechanics

**10.11** Memory math: estimating VRAM needs for QLoRA fine-tuning at 7B/13B/70B

**10.12** Single-GPU fine-tuning workflow end-to-end

**10.13** Alignment beyond SFT: RLHF pipeline (reward model + PPO)

**10.14** Direct Preference Optimization (DPO) — simpler, more stable alternative to PPO

**10.15** ORPO and KTO — further simplifications, when to use which

**10.16** Building preference datasets for DPO/ORPO

**10.17** Evaluating a fine-tuned model against its base model (task-specific benchmarks)

**10.18** Overfitting/catastrophic forgetting diagnosis

**10.19** Cost-benefit review: was fine-tuning worth it vs a strong RAG+prompt baseline?

**10.20** Fine-tuning tooling: Axolotl (YAML-driven configs), Unsloth (2x faster LoRA/QLoRA), LLaMA-Factory (low-code UI)

**10.21** Distributed fine-tuning: DeepSpeed ZeRO stages, FSDP, gradient checkpointing

**10.22** Model merging: SLERP, TIES-merging, DARE, multi-LoRA-adapter merging with MergeKit

## Module 11: Quantization & Inference Optimization

**11.1** Quantization fundamentals: INT8/INT4, symmetric vs asymmetric quantization

**11.2** Post-training quantization (PTQ) vs quantization-aware training (QAT)

**11.3** Calibration datasets and their effect on quantized-model quality

**11.4** Quality/speed/memory trade-off curves across quantization levels

**11.5** GPTQ algorithm deep dive: layer-wise, Hessian-based weight quantization

**11.6** GPTQ calibration process and group-size trade-offs

**11.7** When GPTQ underperforms — outlier weights and their impact

**11.8** Practical GPTQ quantization workflow with AutoGPTQ

**11.9** AWQ (Activation-aware Weight Quantization): theory — why activation magnitude matters more than weight magnitude

**11.10** AWQ vs GPTQ — accuracy and speed comparison

**11.11** GGUF format & llama.cpp quantization levels (Q2_K through Q8_0, Q4_K_M, Q5_K_M) explained

**11.12** Choosing a quantization format for a target deployment (server GPU vs consumer GPU vs CPU/edge)

**11.13** bitsandbytes library deep dive: 8-bit and 4-bit (NF4) inference

**11.14** Mixed-precision inference (fp16/bf16) vs quantized inference

**11.15** Speculative decoding as a complementary speed technique

**11.16** Hardware-specific considerations: consumer GPU (24GB) vs data-center GPU (A100/H100) vs Apple Silicon/CPU